



Práctica de laboratorio 3

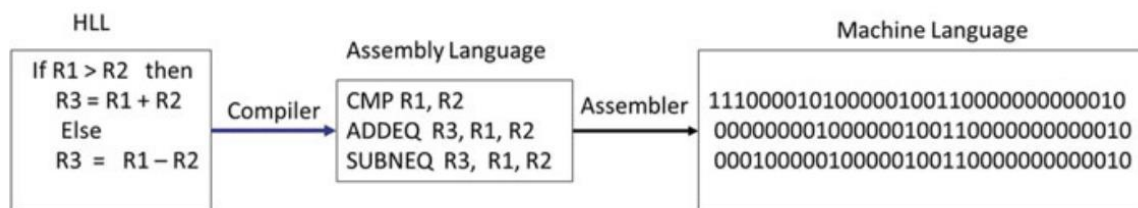
Flujo de trabajo

Objetivo:

Crear un programa simple en lenguaje ensamblador que genera un solo valor para ser mostrado por una línea de comando. Establece un ejemplo del flujo de trabajo (es decir, una serie de líneas de comando GNU / Linux) utilizado en las siguientes prácticas de laboratorio.

Introducción

Los programadores usan lenguaje de alto nivel para desarrollar programas de aplicación; para que el programa se convierta en un formato ejecutable, debe convertirse en código de máquina (binario).



La imagen nos muestra un lenguaje de alto nivel (HLL) convertido a código de máquina, el compilador convierte HLL en lenguaje ensamblador y luego en lenguaje de máquina (bits).

Cada CPU tiene un conjunto de instrucciones que representa el tipo de operaciones que puede realizar, estas instrucciones se representan en formas mnemotécnicas o abreviaturas, por ejemplo, la instrucción de suma se representa con "ADD" y la instrucción de resta se representa con "SUB".

ADD R1, R2, R3 significa sumar el contenido de R2 con el registro R3 y almacenar los resultados en el registro R1. (R1, R2 y R3 se denominan operandos).

Razones para aprender lenguaje ensamblador.

Hay muchos lenguajes de programación de alto nivel, como Java, Python, C y C++ que han sido diseñados para permitir a los programadores trabajar con un alto nivel de abstracción, de modo que no necesiten comprender exactamente qué instrucciones necesita un programador en una CPU particular.

Los lenguajes de alto nivel pueden mejorar enormemente la productividad del programador. Sin embargo, hay algunas situaciones en las que escribir código ensamblador directamente es deseable o necesario. Por ejemplo, el lenguaje ensamblador puede ser la mejor opción al escribir:

- Los primeros pasos para arrancar la computadora.
- Código para manejar interrupciones.
- Código de bloqueo de bajo nivel para programas de subprocesos múltiples
- Código para máquinas donde no existe un compilador.
- Código que necesita ser optimizado más allá de los límites del compilador.
- En computadoras con memoria muy limitada.
- Código que requiere acceso de bajo nivel a características arquitectónicas del procesador.

El lenguaje ensamblador es una parte importante del camino para comprender cómo funciona la máquina. Aunque solo un pequeño porcentaje de informáticos podrán trabajar en el generador de código de un compilador, todos pueden beneficiarse del nivel más profundo de comprensión que se obtiene al aprender el lenguaje ensamblador. Muchos programadores no entienden realmente los punteros hasta que han escrito en lenguaje ensamblador.

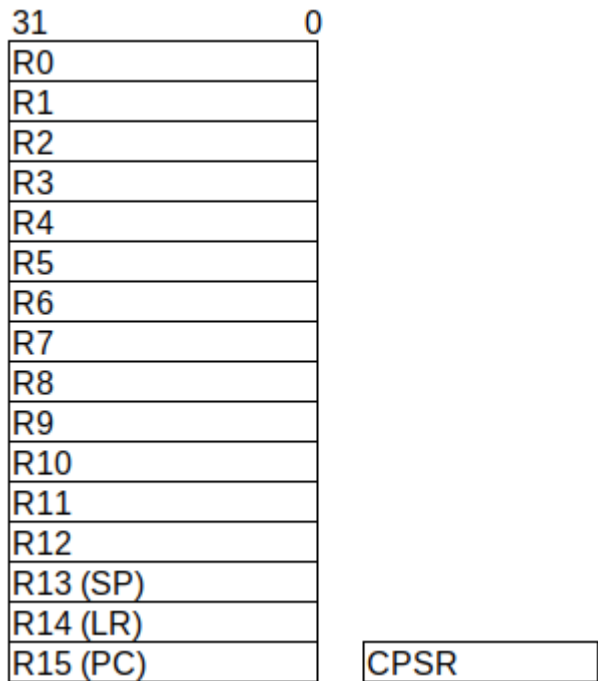
Registros ARM

ARM7 tiene 31 registros generales y 6 registros de estado. En el modo de usuario, solo 16 registros y 1 registro de estado del programa (PSR) están disponibles para los programadores. Los registros están etiquetados de R0 a R15.

El registro SP (R13) es el puntero de la pila. La pila en sí es simplemente un bloque contiguo de memoria que el sistema operativo asigna a un proceso o subproceso. Los programas usan la pila para conservar valores de registro, pasar argumentos de funciones y almacenar datos temporales.

El registro LR (R14) es el registro de enlace. Este registro facilita las llamadas y devoluciones de funciones (subrutinas). Una función también puede usar el registro LR como un registro de uso general siempre que conserve el contenido original en la pila o en otro registro.

El registro PC (R15) es el contador del programa. El registro de PC contiene la dirección de la próxima instrucción que el procesador obtendrá de la memoria.



Estructura básica de un programa

La principal característica de un módulo fuente en ensamblador es que existe una clara separación entre las instrucciones y los datos. De forma general un programa en ensamblador está compuesto por la sección de datos y sección de código.

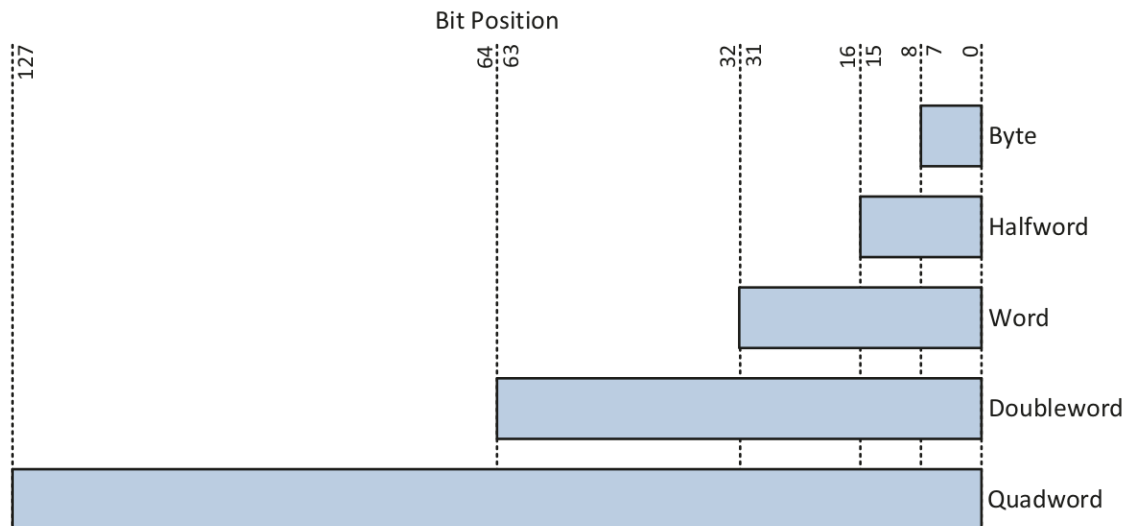
La sección de datos está identificada por la directiva **.data**. En esta sección se definen todas las variables que utiliza el programa con el objeto de reservar memoria para contener los valores asignados. La sección data se suele colocar al principio del programa pero también puede ir al final.

La sección de código se indica con la directiva **.text**, y sólo puede contener código o datos no modificables. Todas las instrucciones son de 32 bits.

De estas dos secciones, **.text** es la única obligatoria

Tipos de datos

ARM	bits	Rango
.byte	8	0 a 255
.hword	16	0 a 65,535
.short	16	-32,768 a 32,767
.word	32	0 a 4294967296
.int	32	-2147483648 a 2147483647
.quad	64	0 a 2 ⁶⁴
	64	-2 ⁶³ a 2 ⁶³ -1



Para representar números utilizamos la información de la siguiente tabla

Tipo	Representación	Ejemplos
Decimal	No lleva delimitador especial	32
Hexadecimal	Prefijo 0x	0xA4
Binario	Prefijo 0b	0b111
Octal	Prefijo 0	012

Para representar una constante utilizamos la directiva `.equ`

`.equ NOMBRE_CONSTANTE, VALOR`

`.equ IVA, 13`

Al compilarse el programa, se sustituirá la palabra 'IVA' por su valor '13'

Formato de las instrucciones

Las instrucciones del ensamblador responden al formato general: Etiqueta, Nemotécnico, Operando u operandos y comentario.

etiqueta: **NEMOTÉCNICO** **Operando/s** **//Comentario**

De estos campos, sólo el nemónico (nombre de la instrucción) es obligatorio, aunque podría ir solo la etiqueta cuando se usa para nombrar secciones del programa. Los campos se separan entre sí por al menos un carácter espacio (ASCII 32) o un tabulador y no existe distinción entre mayúsculas y minúsculas.

Proceso para escribir programas.

Desde la perspectiva del programador, el desarrollo de software es un círculo vicioso de modificar el programa, probar el programa, modificar el programa, probar el programa, modificar el programa, probar el programa hasta que estemos satisfechos con los resultados de la prueba. Un programa en lenguaje ensamblador consta de líneas de texto que crearemos y modificaremos utilizando un editor de texto simple. Luego probamos el programa traduciéndolo al código de máquina ARM para ejecutarlo desde el símbolo del sistema. En esta asignatura, utilizamos la distribución estándar de Raspbian Linux, que incluye el ensamblador y el enlazador GNU que se ejecutan desde la línea de comandos.

El proceso de compilar, vincular y probar debe realizarse desde el símbolo del sistema, pero hay varias formas de editar el programa de código fuente. Si se utiliza Raspberry Pi emulado a través de QEMU es recomendable trabajar con su IDE favorito conectándose vía SSH. O puede ingresar vía SSH y utilizar un editor como nano que se abre desde la línea de comandos.

Desarrollo de la práctica

Utilizar el editor para escribir el siguiente programa

```
GNU nano 3.2                                ejercicio1.s
1      .data                                // Sección de datos
2      .text                                // Sección del código
3      .global _start                       // Definir el punto de entrada
4      _start:                             // Etiqueta
5      MOV    R0, #78                       // Mover el número 78 en el registro R0
6      MOV    R7, #1                       // Código de llamada al sistema, 1 para terminar el programa
7      SWI     0                           // Ejecutar la llamada al sistema, equivalente a SVC
```

Elementos utilizados en la práctica

Instrucciones ARM

MOV R0, #78	Mueve el valor “inmediato” 78 en decimal al registro R0
SWI 0	Realiza una llamada al sistema

Directivas

.data	Sección donde se pondrán las variables, en esta práctica aún no hemos definido variables
.text	Sección que identifica donde irá el código
.global	Se utiliza para identificar el punto de inicio del programa, esta información es utilizada por el enlazador.

Llamadas a servicios del sistema

MOV R7, #1 SWI 0	Realiza la llamada al sistema (código 1 que se carga en el registro 7) para terminar el programa
---------------------	--

Compilar

Una vez editado el archivo, el programa GNU “as” lee las instrucciones del programa en el archivo fuente y escribe un archivo objeto conteniendo los códigos de instrucciones de máquina de la CPU en la que se puede ejecutar. El comando a utilizar será “**as -o ejercicio1.o ejercicio1.s**”. El parámetro -o especifica el archivo de salida y al final va el nombre del archivo de entrada.

Enlazar

El programa “ld”, enlaza el programa leído desde el archivo objeto, lo combina con las librerías necesarias (rutinas de entrada/salida, funciones matemáticas, entre otras) y crea un archivo ejecutable en el lenguaje de máquina de la CPU especificada. El comando será “**ld -o ejercicio1 ejercicio1.o**” el archivo de salida será ejercicio1 y el de entrada es ejercicio1.o

Ejecutar

Para ejecutar el archivo colocamos en la consola “./ejercicio1”. Por la forma en que se ha realizado el ejercicio no mostrará nada, puesto que este coloca el valor 78 en el registro R0, por convención el

valor retornado por las funciones debe estar en el registro R0 y podemos ver su valor con el comando “**echo \$?**”

El flujo de instrucciones es el siguiente

```
pi@raspberrypi:~/asm/ejercicios $ nano ejercicio1.s
pi@raspberrypi:~/asm/ejercicios $ as -o ejercicio1.o ejercicio1.s
pi@raspberrypi:~/asm/ejercicios $ ld -o ejercicio1 ejercicio1.o
pi@raspberrypi:~/asm/ejercicios $ ./ejercicio1
pi@raspberrypi:~/asm/ejercicios $ echo $?
78
```

Utilizar GNU GCC

Para utilizar las herramientas de GNU GCC se debe hacer un cambio, la línea “.global _start” debe cambiar a “.global main”. Puesto que este es el estándar para esta herramienta. Ahora podemos compilar y enlazar en el mismo paso.

```
pi@raspberrypi:~/asm/ejercicios $ nano ejercicio1.s
pi@raspberrypi:~/asm/ejercicios $ gcc -o ejercicio1 ejercicio1.s
pi@raspberrypi:~/asm/ejercicios $ ./ejercicio1
pi@raspberrypi:~/asm/ejercicios $ echo $?
78
pi@raspberrypi:~/asm/ejercicios $
```

Ejercicios y preguntas de revisión

1. Utiliza el emulador en línea para ejecutar el programa de la práctica
<https://cpulculator.01xz.net/?sys=arm-de1soc>
2. ¿Cuál es la diferente entre código fuente y código objeto?
3. Modifica el programa removiendo la línea “.global _start”. Luego compile, enlace y ejecute como antes. ¿En qué etapa se capturó el error?
4. Modifique el programa anterior y quite el carácter “#” (78 en lugar de #78). ¿Qué ocurre cuando se compila, enlaza y ejecuta?
5. Analice el siguiente programa en C

```
1 #include <stdio.h>
2 static char str[] = "Hola mundo\n";
3 int main() {
4     printf(str);
5     return 0;
6 }
```

Escriba, compile y ejecute el siguiente programa en ensamblador que es equivalente al presentado en C. Compare y analice la estructura de los programas.

```

1      .data
2  str: .asciz "Hola mundo\n"      @Define un cadena con terminación null
3
4      .text
5      .globl main
6      /* Este es el principio de la función main().
7         se mostrará el mensaje "Hola mundo" .
8      */
9
10     main:
11     STMFD sp!,{lr}      // Colocar en la pila (push) la dirección de retorno
12     LDR    R0, =str      @ Cargar un puntero a la variable str
13     BL     printf      // ejecutar printf("Hola mundo\n");
14     MOV     R0, #0      // Mover el código de retorno en R0
15     LDMFD  sp!,{lr}      @ Extraer de la pila (pop) la dirección de retorno
16     MOV     pc, lr      // Equivalente a return 0; desde la función main

```


Bibliografía

- Elahi, Ata. Computer Systems. Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language. Segunda Edición. 2022.
- Pyeatt, Larry. Modern Assembly Language Programming with the ARM Processor. 2016.